

第4章 软件架构的风格与模式

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第4章 软件架构的风格与模式

- 4.0 引言
- 4.1 软件架构风格定义
- 4.2 软件架构风格的分类
- 4.3 典型的软件架构风格
- 4.4 软件架构模式

4.2 软件架构风格的分类

- (1) 数据流风格：批处理、管道/过滤器；
- (2) 调用/返回风格：主程序/子程序、面向对象风格、层次结构；
- (3) 以数据为中心的风格：仓库风格、超文本系统、黑板风格等；
- (4) 虚拟机风格：解释器、基于规则的系统；
- (5) 独立组件风格：进程通讯、事件系统。

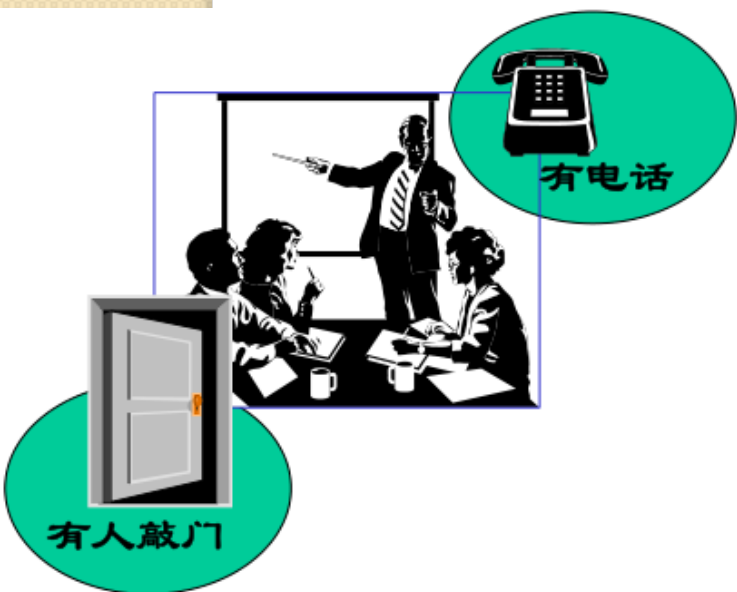
4.3.5 事件驱动风格

- 消息

- 计算机中，消息是具有特定含义的数据
- 对象通过发送消息的方式请求另一对象为其服务。

- 事件

- 能够激活对象功能的动作。当发生这种动作后将给所涉及对象发送一个消息，对象便可执行相应的功能

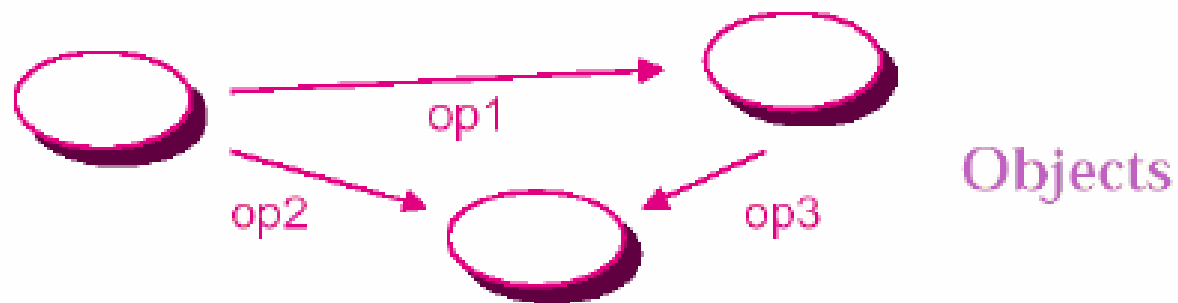


4.3.5 事件驱动风格

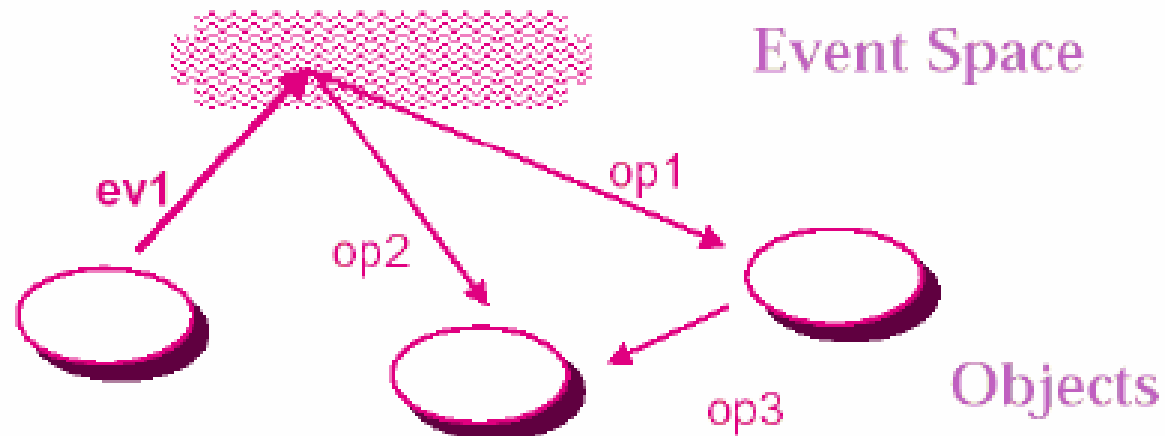
- 通常，在一个系统中，组件接口提供了访问过程或函数的端口的集合，组件通过显式地调用这些过程或函数来与其他组件交互。
- 然而，一种基于隐式调用(implicit invocation)的集成技术非常受关注，该技术就是事件驱动(Event-based) 的软件架构风格。

显式调用 vs. 隐式调用

Explicit Invocation



Implicit Invocation

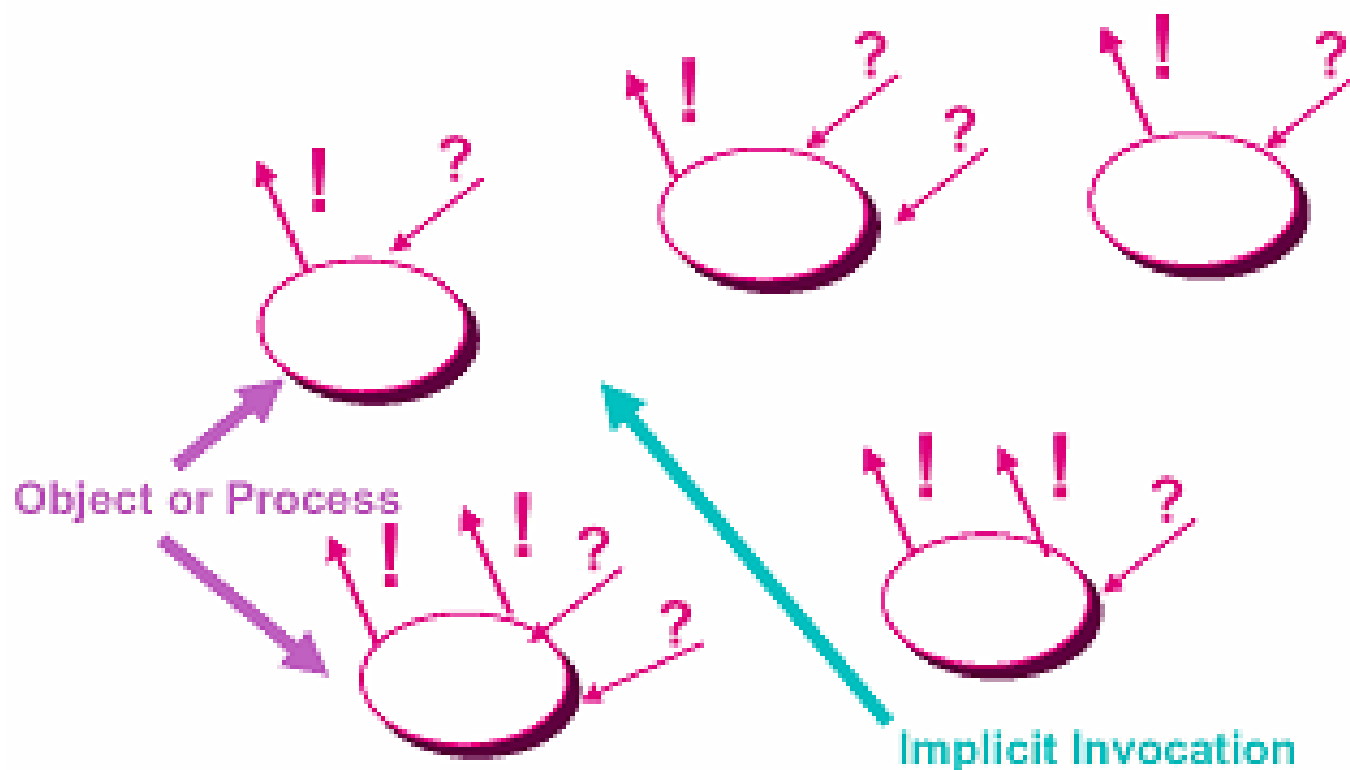


显式调用 vs. 隐式调用

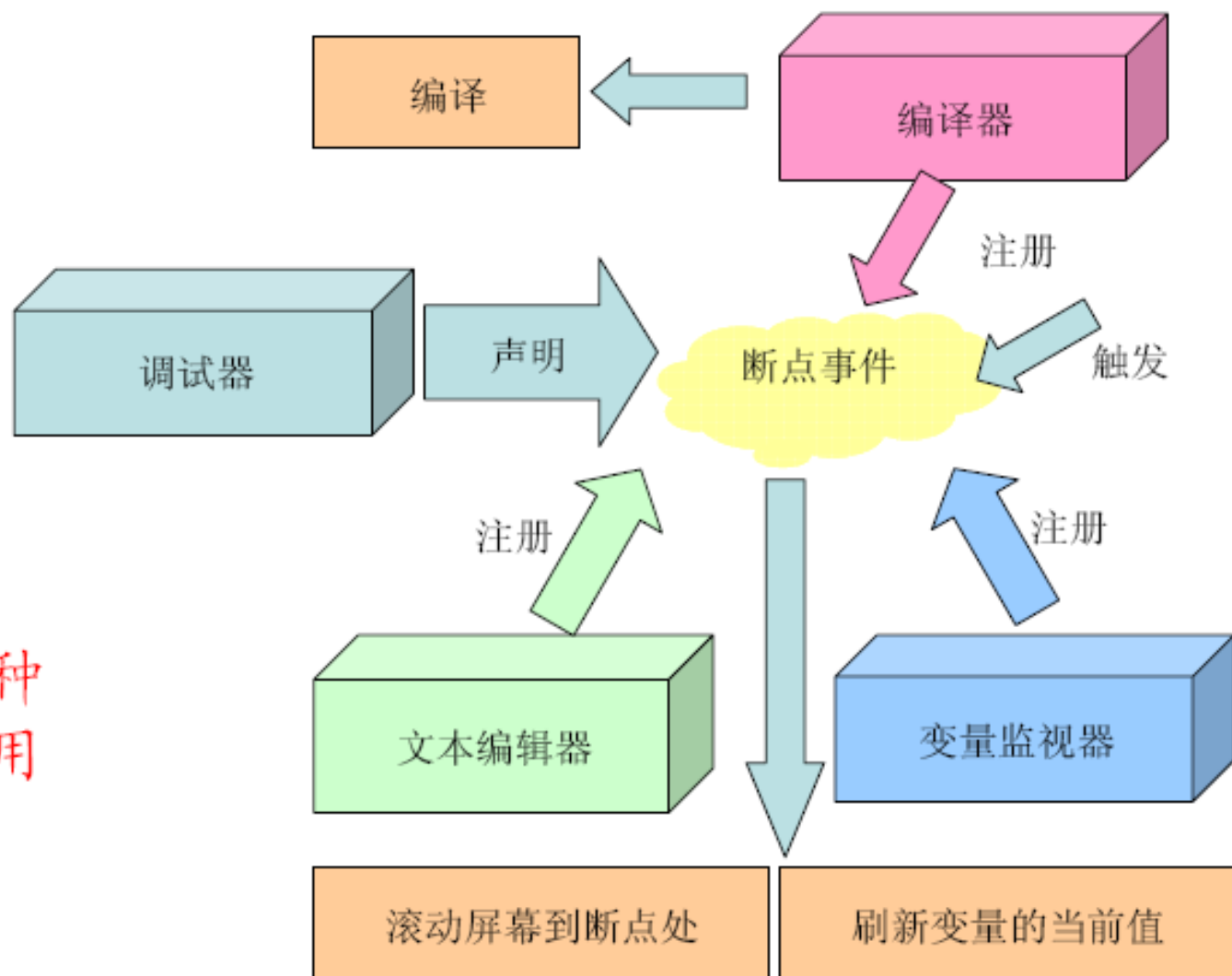
- Traditionally, in a system in which the component provide a collection of routines and functions, such a an object-oriented system, components typically interact with each other by explicitly invoking those routines.
- 各个构件之间的互动是由显性调用函数或程序完成的。
- 调用过程与次序是固定的、预先设定的。

显式调用 vs. 隐式调用

- 在很多情况下，软件更多地变成被动性系统，构件持续地与其所处的环境打交道，但并不知道确切的交互次序



显式调用 vs. 隐式调用

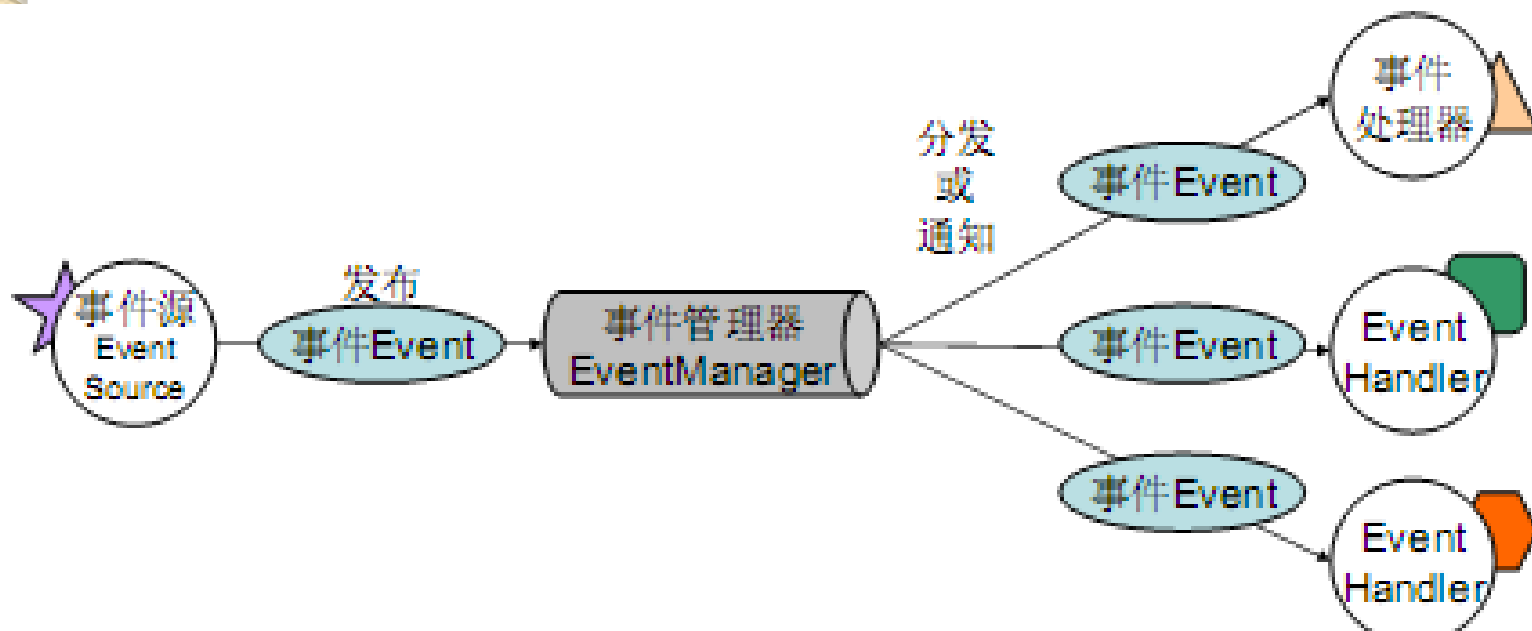


这是一种
隐式调用

4.3.5 事件驱动风格

- 基本思想：
 - 组件不直接调用一个过程，而是发布或广播一个或多个事件。
 - 系统中的其它组件通过注册与一个事件关联起来的过程，来表示对某一个事件感兴趣。当这个事件发生时，系统本身会调用所有注册了这个事件的过程。这样一个事件的激发会导致其它模块中过程的隐式调用。
 - 事件：是指可以被系统识别的、发生在界面上的用户动作或者状态变化。

4.3.5 事件驱动风格



功能	描述
分离的交互	事件发布者并不会意识到事件订阅者的存在。
多对多通信	采用发布/订阅消息传递，一个特定事件可以影响多个订阅者。
基于事件的触发器	控制流由接收者确定（基于发布的事件）。
异步	通过事件消息传递支持异步操作。

4.3.5 事件驱动风格

- 这种风格的主要特点是：事件的触发者并不知道哪些构件会被这些事件影响，相互保持独立。
 - 这样不能假定构件的 处理顺序，甚至不知道哪些过程会被调用；
 - 各个构件之间彼此无直接的 连接关系，各自独立存在，通过对事件的发布和注册实现关联。

事件调度策略

Event Systems
(事件系统)

EventManager with
separated dispatcher module
(有独立的事件分派模块)

EventManager without
separated dispatcher module
(无独立的事件分派模块)

All broadcasting
(全广播)

Selected broadcasting
(选择性广播)

Point-to-Point
(Queue-based)
点对点(基于队列)

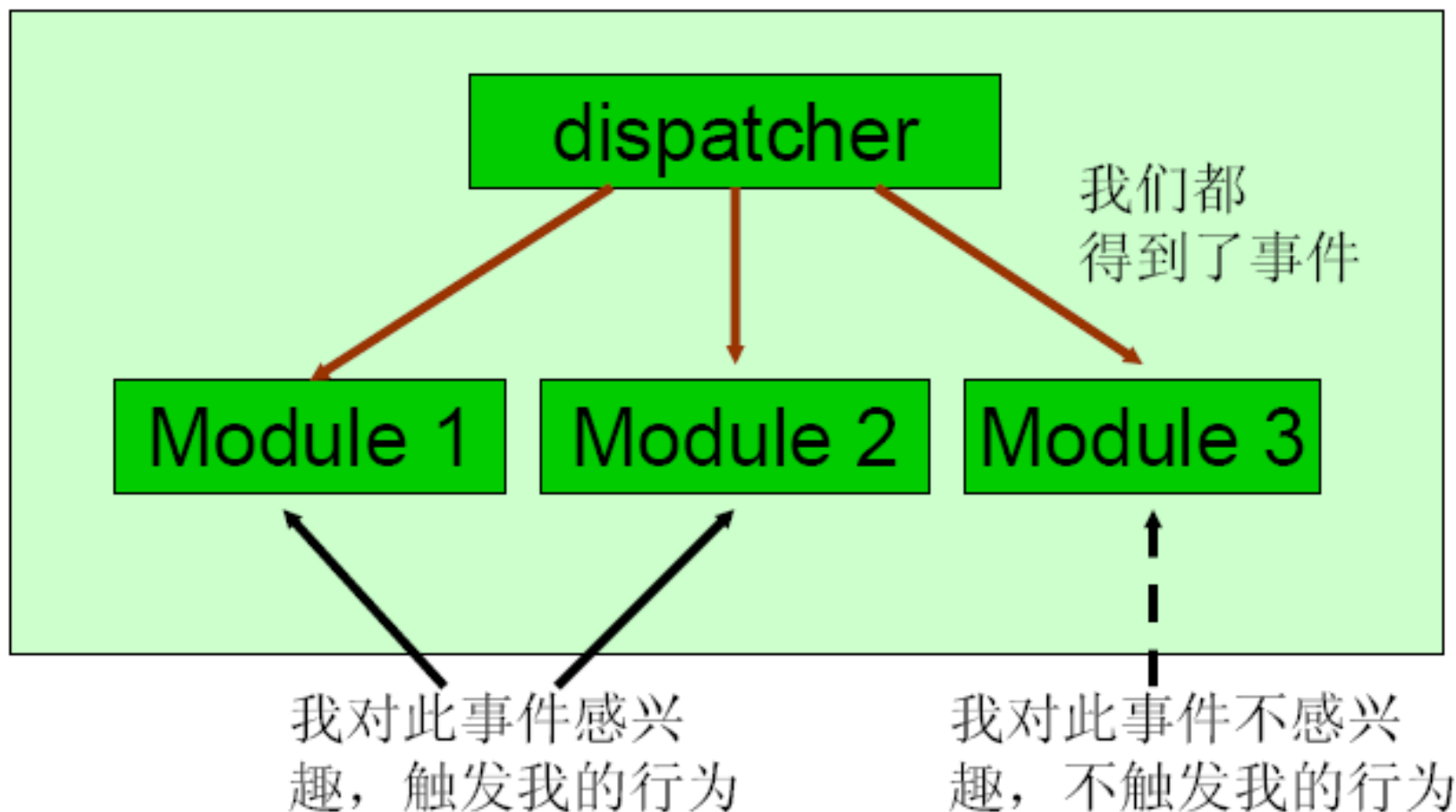
Publish-Subscribe
(发布-订阅)

事件调度策略

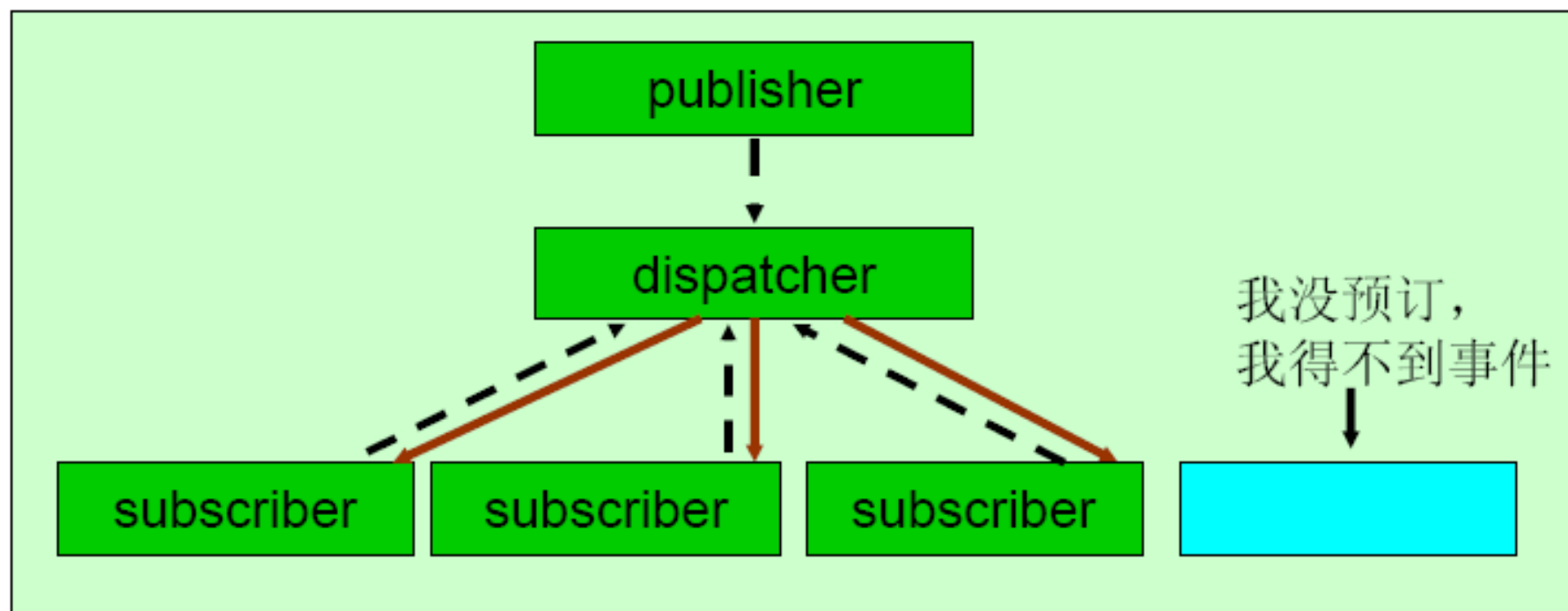
- 事件分派模块的功能
 - 负责接收到来的事件并派遣它们到其它模块
- 一般分为两种分派策略
 - 广播式(All broadcasting)：派遣模块将事件广播到所有的模块，但只有感兴趣的模块才去取事件并触发自身的行为；
 - 选择广播式(Selected broadcasting)：派遣模块将事件送到那些已经注册了的模块中。

广播式 (All broadcasting)

无目的广播，靠接受者自行决定是否加以处理或者简单抛弃



选择广播式(Selected broadcasting)



— . — . — . ▶

Publish event

.....▶

Subscribe

————▶

Send event

有目的广播，
只发送给那些已经注册过的订阅者

事件调度策略

Event Systems
(事件系统)

EventManager with
separated dispatcher module
(有独立的事件分派模块)

EventManager without
separated dispatcher module
(无独立的事件分派模块)

All broadcasting
(全广播)

Selected broadcasting
(选择性广播)

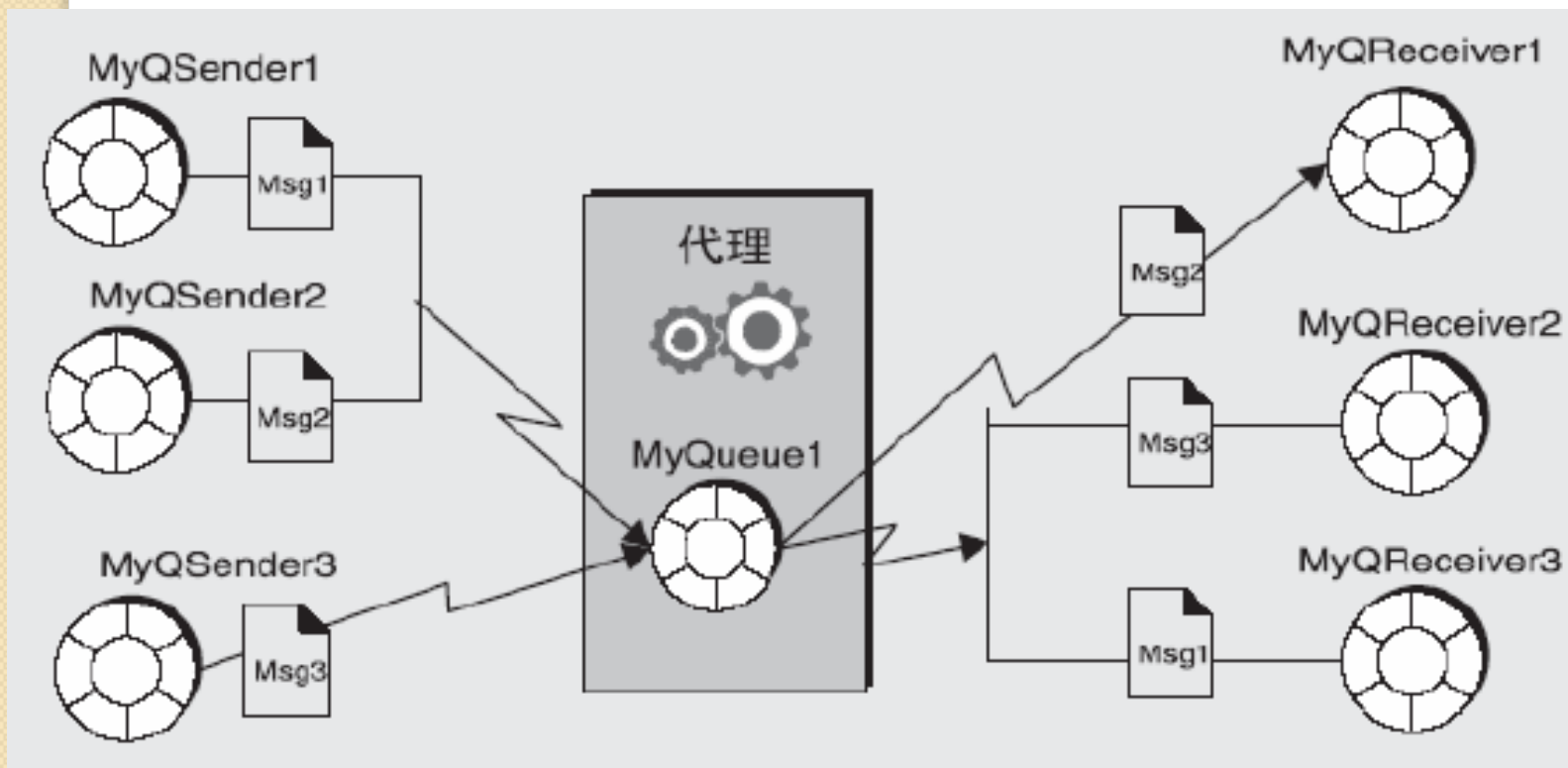
Point-to-Point
(Queue-based)
点对点(基于队列)

Publish-Subscribe
(发布-订阅)

点对点的选择广播式：消息队列 Point-to-Point (Message Queue)

- 系统安装并配置一个队列管理器，并定义一个命名的消息队列
- 某个应用向消息队列注册，以监听并处理被放置在队列里的事件
- 其他的应用连接到该队列并向其中发布事件
- 队列管理器存储这些消息，直到接收端的应用连接到队列，取回这些消息并加以处理
- 消息只能够被唯一的消费者所消费，消费之后即从队列中删除

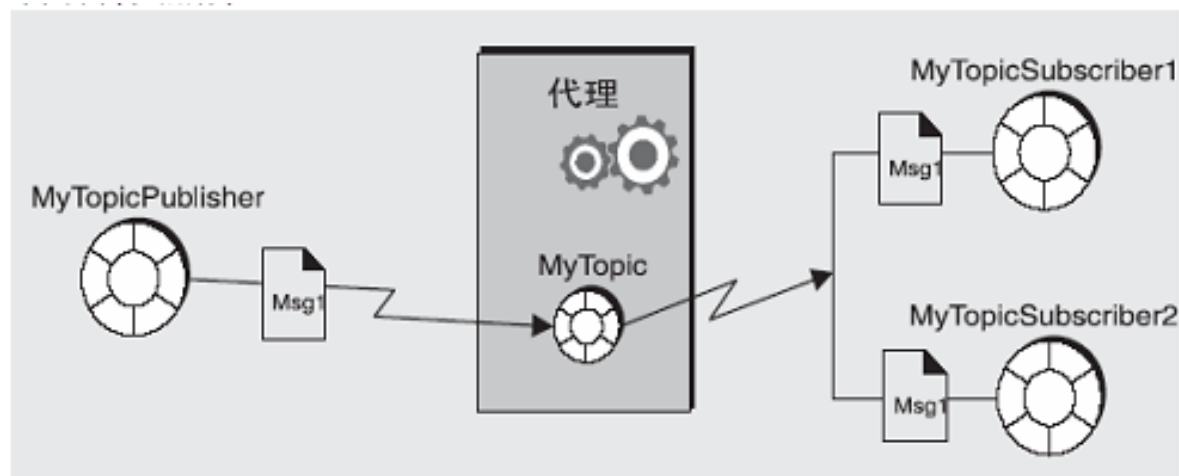
- (更复杂的)点对点的选择广播式



发布-订阅的选择广播式

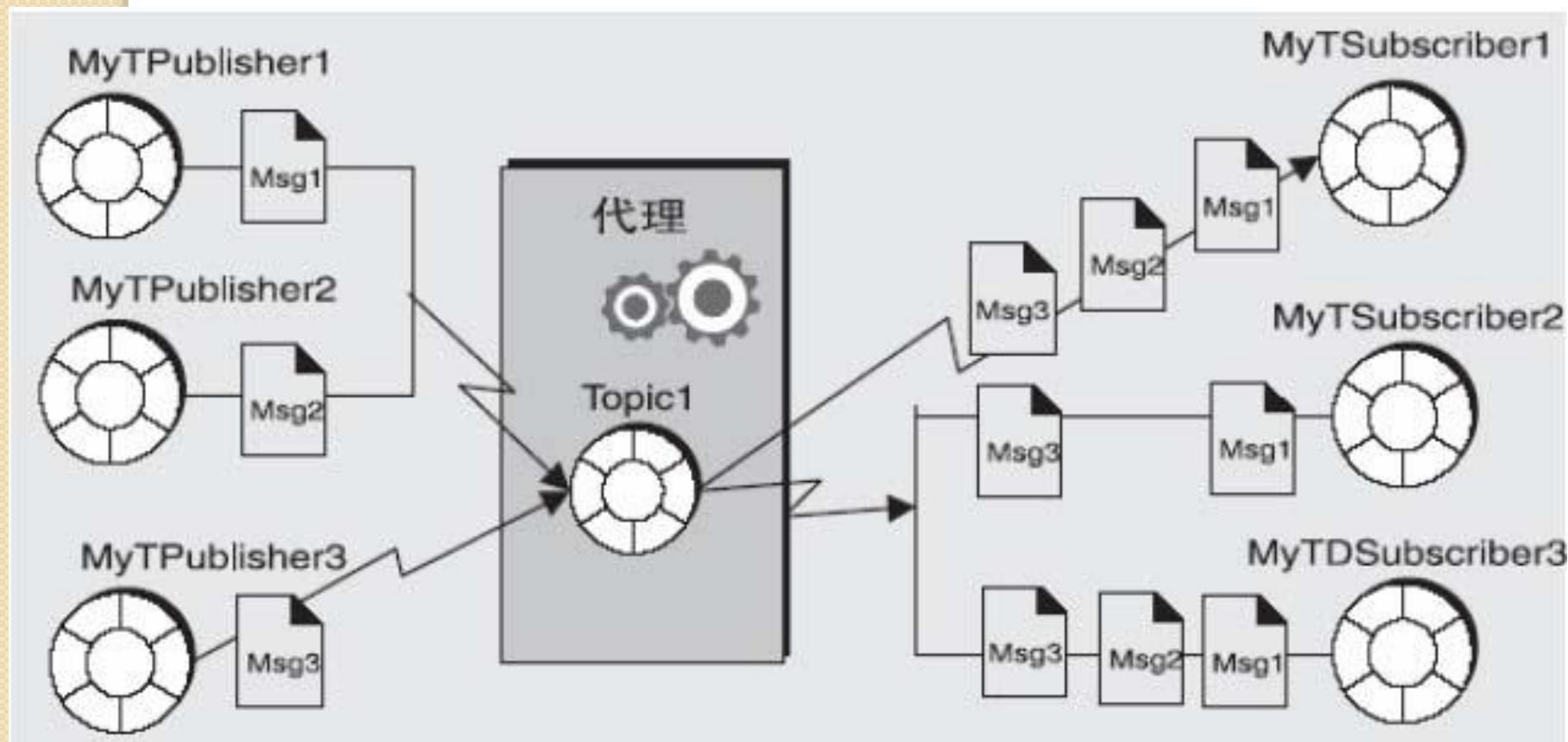
Publish-Subscribe

- 事件发布者向“Topic（主题）”发布事件，订阅者向“主题”订阅事件
- 一个事件可以被多个订阅者消费；
- 事件在发送给订阅者之后，并不会马上从topic中删除，topic会在事件过期之后自动将其删除。



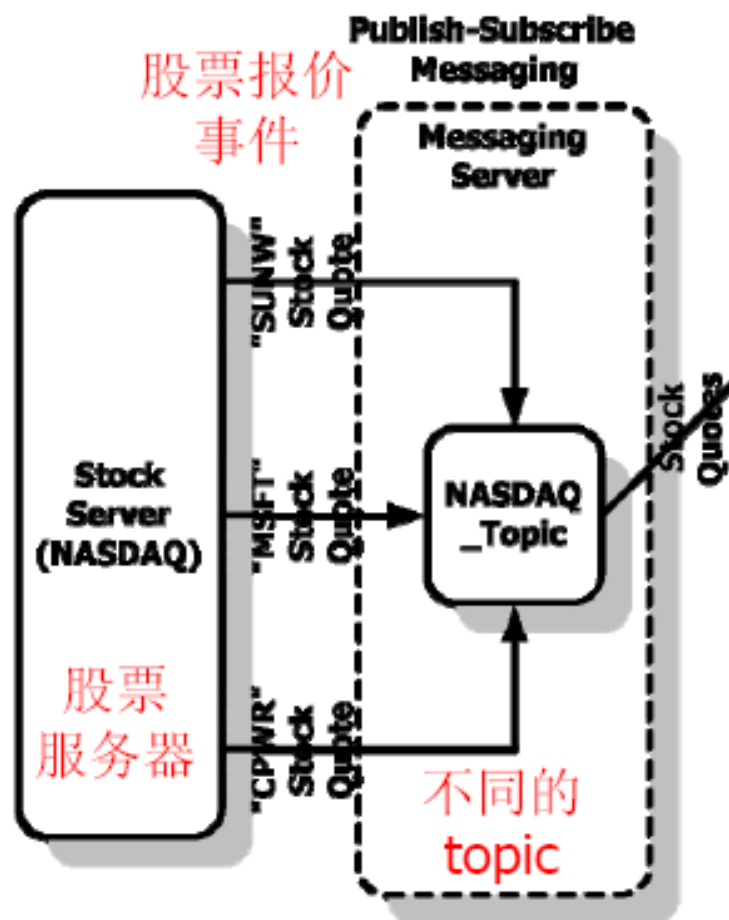
生活中的例子：新闻订阅/RSS

- (更复杂的)发布-订阅的选择广播式

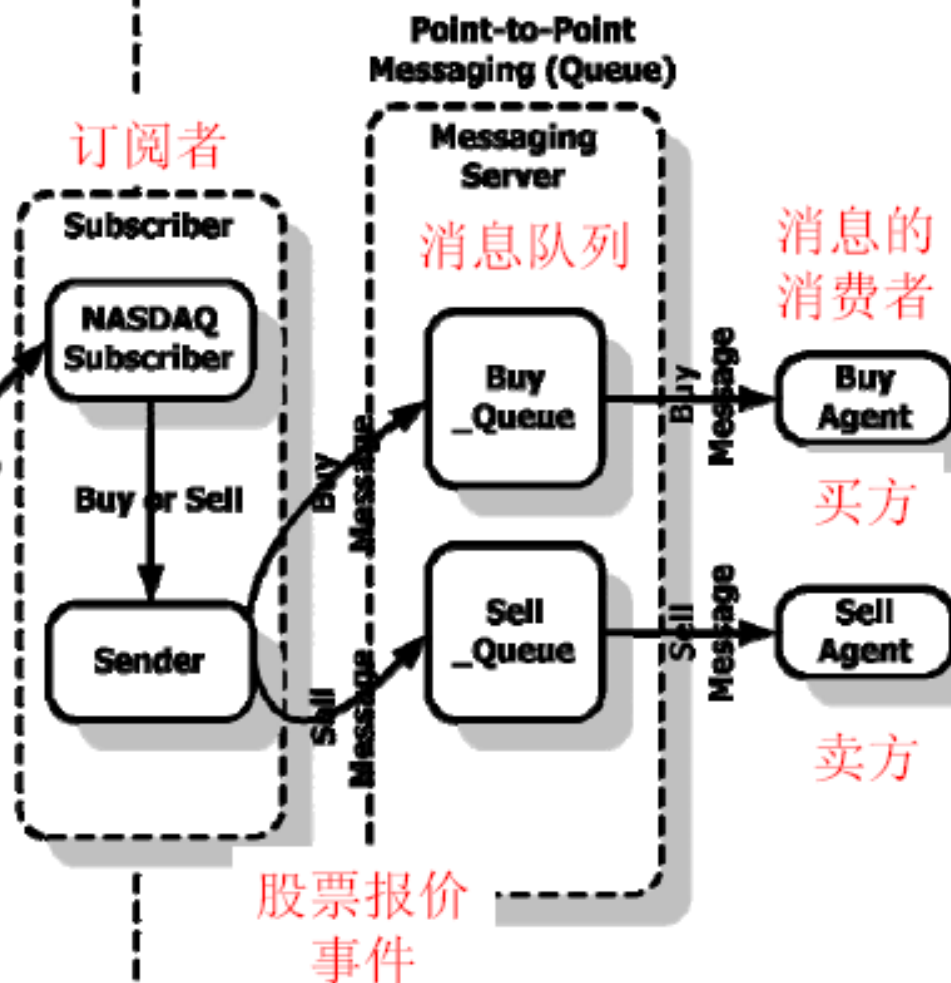


股票交易市场

发布-订阅模式



点对点模式



事件调度策略

Event Systems
(事件系统)

EventManager with
separated dispatcher module
(有独立的事件分派模块)

EventManager without
separated dispatcher module
(无独立的事件分派模块)

All broadcasting
(全广播)

Selected broadcasting
(选择性广播)

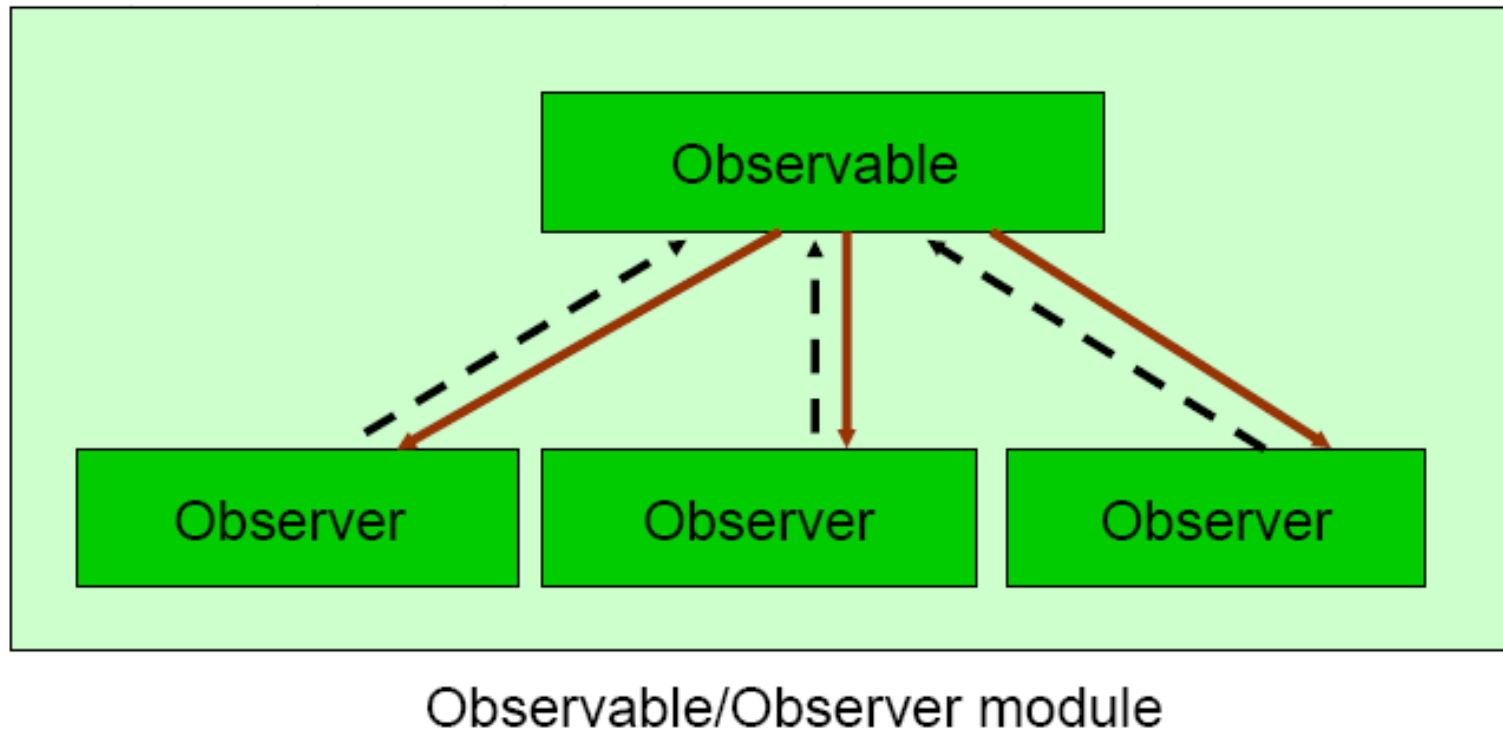
Point-to-Point
(Queue-based)
点对点(基于队列)

Publish-Subscribe
(发布-订阅)

无独立调度模块的事件系统

- This module is usually called Observable/Observer (称为“被观察者/观察者”).
- 每一个模块都允许其他模块向自己所能发送的某些消息表明兴趣
- 当某一模块发出某一事件时，它自动将这些事件发布给那些曾经向自己注册过此事件的模块

无独立调度模块的事件系统



Legend: ➡ Register event ➡ Send event

4.3.5 事件驱动风格

- 事件驱动编程的一般步骤：
 - 1) 确定响应事件的元素；
 - 2) 为指定元素确定需要响应的事件类型；
 - 3) 为该元素的相应事件编写事件处理程序；
 - 4) 将事件处理程序绑定到指定元素的指定事件。

4.3.5 事件驱动风格

- 应用实例：Field系统

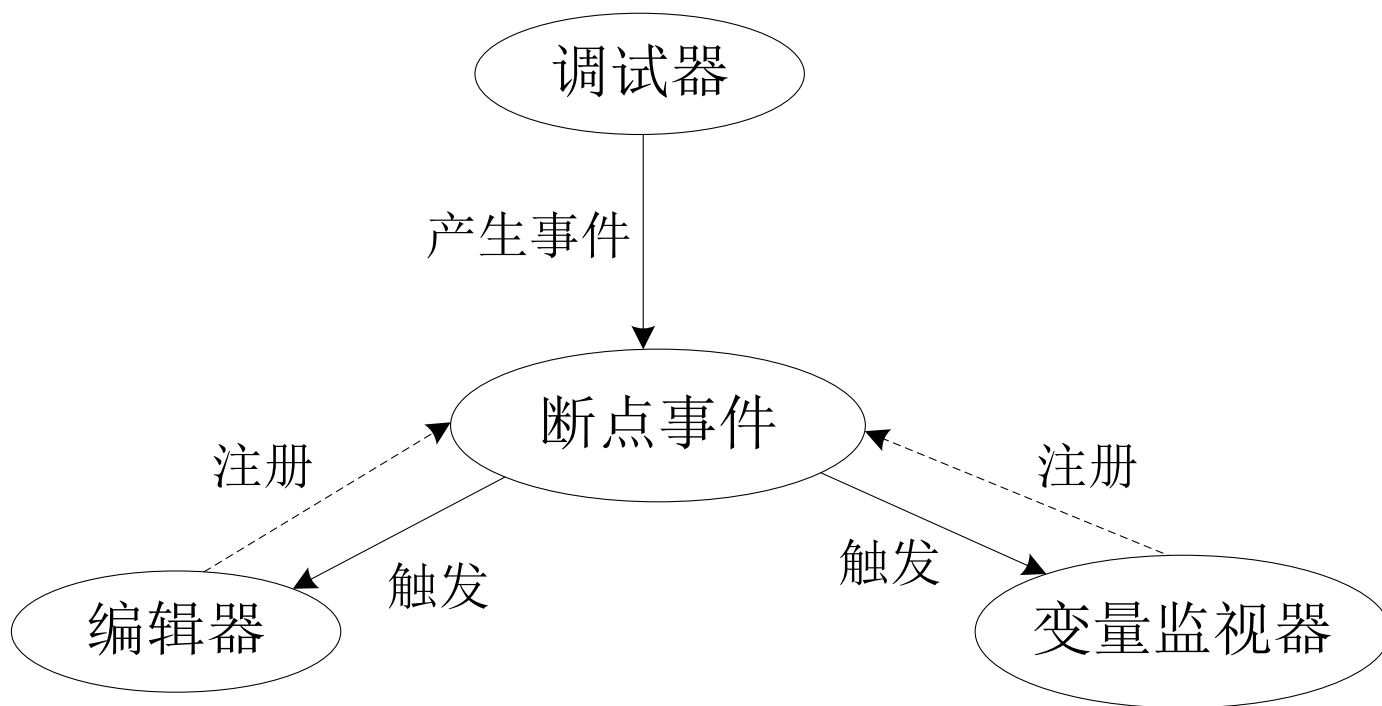
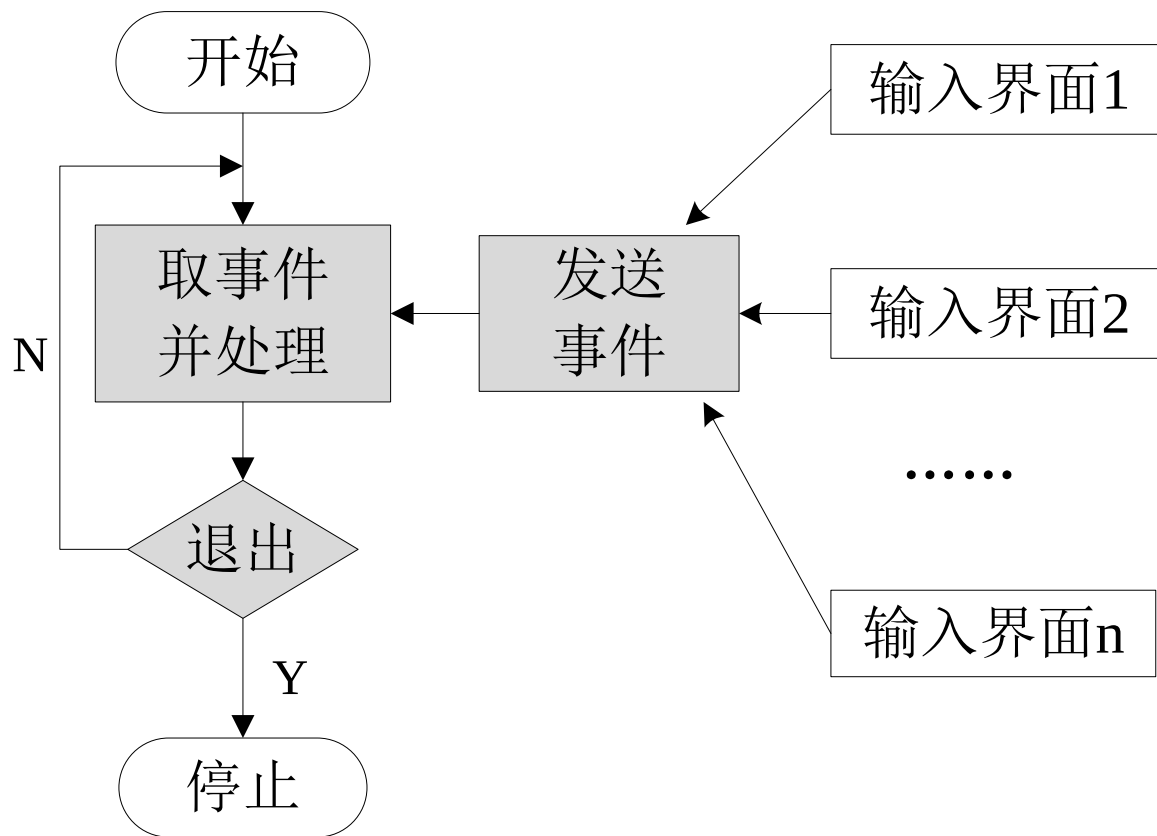


图4.12调试器中的断点处理

4.3.5 事件驱动风格

- 应用实例： Win32 GUI程序



Win32 GUI 程序

- Windows是事件驱动（消息驱动）的OS，也是基于消息的OS。
- Windows 应用程序中消息有两种送出途径:直接和排队。
 - Windows或某些运行的应用程序可直接发布消息给窗口过程，或者，消息可送到消息队列，在应用程序执行期间应用程序对象连续不断轮询消息队列的消息。凡是以排队方式送出的消息都被送到一个由操作系统提供的消息队列的保留区。在OS中当前执行的每个进程都有各自的消息队列。
- 事件驱动程序不是由事件的顺序来控制，而是由事件的发生来控制，而事件的发生是随机的、不确定的，这就允许程序的用户用各种合理的顺序来安排程序的流程。
- 事件驱动围绕消息的产生与处理展开，它是靠消息循环机制来实现的，消息是一种报告有关事件发生的通知。

4.3.5 事件驱动风格

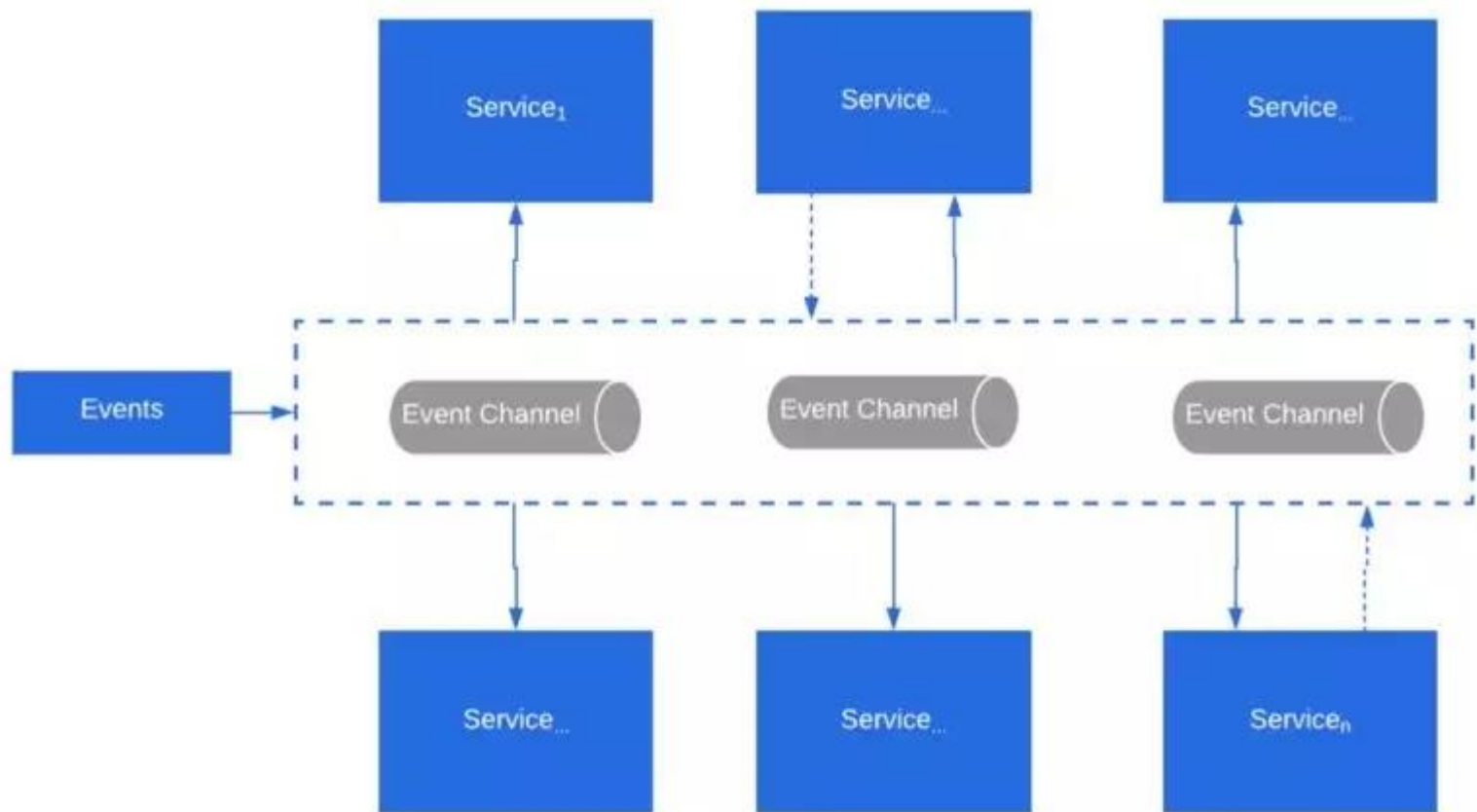


Figure 2 - Broker Topology

4.3.5 事件驱动风格

- 优点

- (1) 事件声明者不需要知道哪些组件会影响事件，组件之间关联较弱。一个组件出错将不会影响到其他构件。
- (2) 提高软件复用能力。只要在系统事件中注册组件的过程，就可以将该组件集成到系统中。
- (3) 系统便于升级。只要组件名和事件中所注册的过程名保持不变，原有组件就可以被新组件替代。

4.3.5 事件驱动风格

- 缺点
- (1) 组件放弃了对计算的控制权，完全由系统来决定。
- (2) 存在数据交换问题。
- (3) 该风格中，正确性验证成为一个问题。



THANKS